

INTRODUCTION TO

ALGORITHM ANALYSIS

ALGORITHM :-

- It is a step by step process.
- Constraints are time and space.

problem
↓

Algorithm
↓

Input → Computer → Output.

Characteristics of Algorithm :-

1. Input
2. Output
3. Finiteness
4. Definiteness
5. Effectiveness

Ways to Analyse Algorithm :-

1. Space Complexity / analysis.
2. Time Complexity / analysis.

SPACE COMPLEXITY :-

- Space complexity = fixed part + variable parts.
- It is the no. of space a program required for it to run.

Eg:- 1) Sum(A, n)

{

for (i=0 to n)

 A[i] = 0

}

A) Space for i=1

1) n=1

 A[i] = n+1

Total = 3 + n Space

$n + 1 + 1 + 1 = n + 3 \Rightarrow 3 + n$

2) abc(a, b, c)

{

 d = a + b * c + (a + b + c) / (a + b) * 4

 print(d)

}

A) Variables = a, b, c, d

Total = 4

3) abc(a, b, c)

{

 return a * b * c + (a + b + c) / (a + b) * 4

}

A) Total = 3

4) for (i=0; i<n; i++)
for (j=0; j<m; j++)
c[i][j] = a[i][j] + b[i][j]

Ans:- $a[i][j] = n \times n = n^2$
 $b[i][j] = n^2$
 $c[i][j] = n^2$

i=1
j=1
n=1
 $\rightarrow 2+3n$

Total = $3n^2 + 3$

TIME COMPLEXITY :-

- Total time required for an algorithm.
- Finding frequency count.

Rules :-

1. Command & declaration has frequency count = 0
2. Return & Assignments Statement has frequency count = 1
3. for looping statement frequency count = no. of times the loop is repeating.
4. Ignore lower order exponents when higher order exponents are present & also ignore

Constant multipliers.

eg:- $(5n^4) + 7n^3 + 10n^2 + n + 100$

time complexity = $O(n^4)$

- Take the variable with highest exponent and ignore the coefficient.

1. Sum() — 0

< — 0

$a = 5$

$b = 6$

$c = a + b$

> >

ans:- 3

growth in

Elementary operation

$O(1)$

Time Requirement is constant and it is independent of the problem's size.

$O(n)$

Time Requirement for a linear algorithm $\uparrow$ directly with the size of problem.

$O(n^2)$ Time Requirement for Quadratic algorithm $\uparrow$ as rapidly with the size of problem.

$O(2^n)$ size of problem $\uparrow$, the time requirement for an exponential algorithm.

$O(n \log n)$ Time requirement $\uparrow$ rapidly than a linear Algorithm.

$O(\log_2 n)$ Time Requirement for a logarithmic algorithm slowly as the problem size $\uparrow$.

2. $abc(a, b, c) = 0$
 $\{$
 $\quad \text{return } a + b \times c + (a + b + c) / (a + b) \times 4 - 1$
 $\}$
 $\text{ans} := O(1)$

3. $\text{Sum}(A, n) \rightarrow 0$
 $\{$
 $\quad S = 0 \rightarrow 1$
 $\quad \text{for } (i = 0; i \leq n; i++) \rightarrow n + 1 \quad (i = 0)$

$s = s + A[i] \rightarrow n - 1$

returns $\rightarrow 1$

}

ans:- $2n + 3 \rightarrow O(n)$

4. $Add(A, n) \rightarrow 0$

{

for($i=0; i < n; i++$) $\rightarrow n+1$

{

for($j=0; j < n; j++$) $\rightarrow n(n+1) = n^2 + n$

}

$s[i][j] = a[i][j] + b[j][j] \rightarrow n^2$

}

}

}

$n^2 + n \rightarrow 2(n^2 + 2n + 1)$
 $\rightarrow O(n^2)$

ans:- $2n^2 + 2n + 1$

$\rightarrow O(n^2)$

5. matmul() - 0

{

for($i=0; i < n; i++$) $\rightarrow n+1$

{

for($j=0; j < n; j++$) $\rightarrow n(n+1) \rightarrow n^2 + n$

{


```

c[i][j] = 0 → n²
for (k=0; k<n; k++) → n²(n+1)
{
    c[i][j] = a[i][k] + c[i][j] + b[k][j]
}
}
}
}
ans:- 2n³ + 3n² + 2n + 1
      ⇒ O(n³)

```

$$\begin{array}{r}
 n+1 \\
 n^2 + n \\
 n^2 \\
 n^2 + n^2 \\
 n^2
 \end{array}$$

$$\begin{array}{r}
 n^3 \\
 n^2 + n \\
 n^2 \\
 n^2 + n^2 \\
 n^2
 \end{array}$$

$$\begin{array}{r}
 2n^3 + 3n^2 + 2n + 1 \\
 \hline
 2n^3 + 3n^2 + 2n + 1
 \end{array}$$

6. Sum(a[], n, m) → 0

```

{
    for(i=0; i<n; i++) → n+1
    {
        for(j=0; j<m; j++) → n(m+1)
        s = s + a[i][j] → nm
    }
    return s → 1
}

```

$$\begin{array}{r}
 n+1 \\
 nm + n \\
 nm \\
 1
 \end{array}$$

$$\begin{array}{r}
 2nm + 2n + 2 \\
 \hline
 2nm + 2n + 2
 \end{array}$$

$$\begin{array}{r}
 \rightarrow \\
 \underline{\underline{O(nm)}}
 \end{array}$$

Analysing Complexity in 3 cases

1. Best case
2. Worst case
3. Average case

- no. of step $\downarrow$, complexity $\downarrow$ so best case.
- no. of step $\uparrow$, so worst case.
- We required only an average no. of cases then it is ^{Average} ~~worst~~ case.

eg:- search(a, n, x) — 0

{ for(i=1 to n) $\rightarrow n+1$

if (a[i] == x) $\rightarrow n$

return i $\rightarrow 0$

return -1 $\rightarrow 1$

} $2n+2$ $O(n)$

Best case is represented using $\Omega(n)$

- Complexity of linear search = $\Omega(1)$.
- ~~If the searching element of array is best case,~~
- If searching element is same as first element of array then best case.

• If middle element is same as searching element then average case.

• of i/p n with another fn called c. g(n).

(For all the i/p greater than no. Then we say that the f(n) is upper bound with $f(n) \leq c \cdot g(n)$ with the multiple of constant c. That is $f(n) = O(g(n))$).

$$O = f(n) \leq c \cdot g(n)$$

$$\Omega = f(n) \geq c \cdot g(n)$$

Asymptotic Notation :-

- The mathematical notations used to decide the running time of an algorithm.
- Asymptotic notations are about hand way to express the least case, avg case and worst case complexities of an algorithm.

Different types of Asymptotic Notation

1. Big Oh Notation (O)
2. Big Omega (Ω)
3. Big Theta (Θ)
4. Little Omega (ω)
5. Little Oh (o)

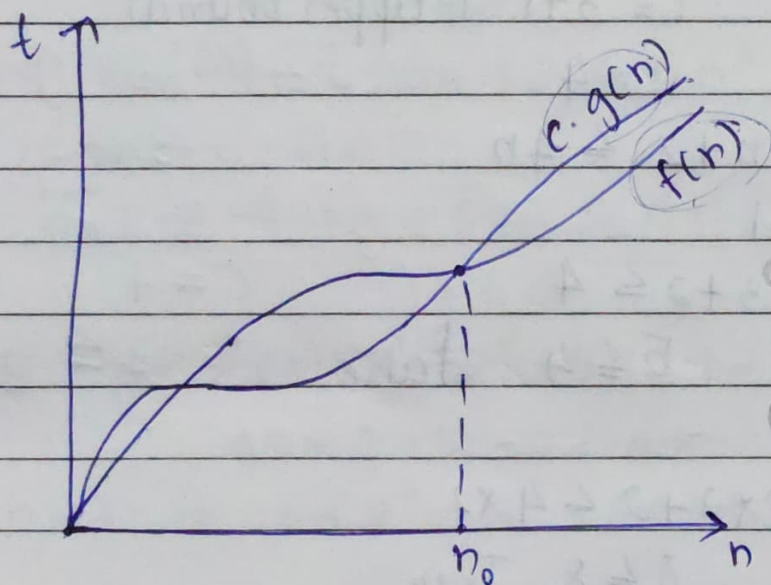
1. Big Oh Notation (O)

For Non negative function $f(n)$ and $g(n)$, if there exist an integer ' n_0 ' and a constant ' c ', $c > 0$ such that for all integer $n \geq n_0$.

$$0 \leq f(n) \leq c \cdot g(n)$$

$$\Rightarrow f(n) = O(g(n))$$

$$\Rightarrow f(n) \leq c \cdot g(n) \quad (c > 0, n \geq n_0)$$



$O(1)$ - constant
 $O(n)$ - linear
 $O(n^2)$ - Quadratic
 $O(n^3)$ - cubic
 $O(2^n)$ - exponential

n - input size
 t - time

It is used to find max amount of time required to complete algorithm.

Upper bound $f(n)$ is represented by big O or worst case.

If we find the time complexity of a given algorithm 'inputs of input n ' with another $f(n)$ called $c \cdot g(n)$ for all input size greater than n_0 , then we say that the $f(n)$ is upper bound with $f(n) = O(g(n))$ with multiple of some constant ' c ' that is $f(n) = O(g(n))$.

~~Imp~~
2.

Big Omega Notation (Ω)

- It is used to represent Best case.
- It is also called Base case Complexity.
- For non negative functions $f(n)$ and $g(n)$, if there exists an integer n_0 and a constant $c > 0$ such that for all

$$f(n) \quad 0 \leq c \cdot g(n) \leq f(n) \times$$

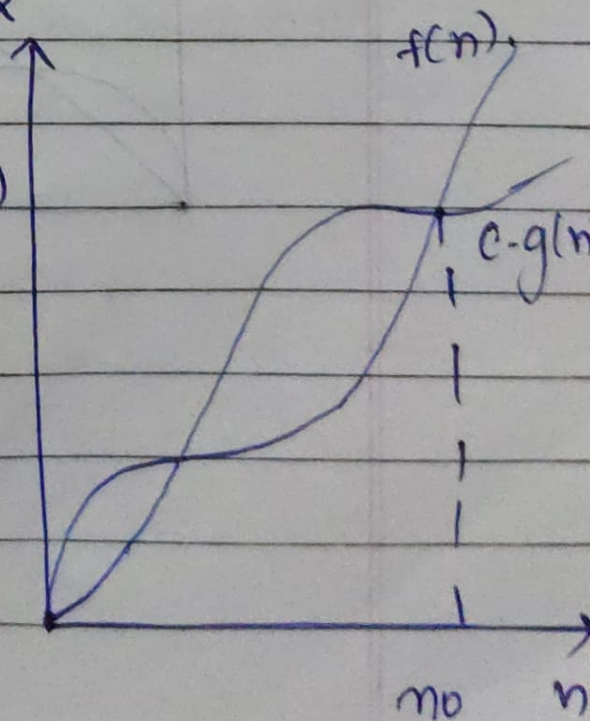
$$\Rightarrow f(n) = \Omega(g(n))$$

$$\Rightarrow f(n) \geq c \cdot g(n)$$

$$f(n) \rightarrow 0 \leq c \cdot g(n) \leq f(n)$$

$$f(n) = \Omega(g(n))$$

$$f(n) \geq c \cdot g(n)$$



$$c_1 g(n) \leq f(n) \leq c_2 g(n)$$

UB

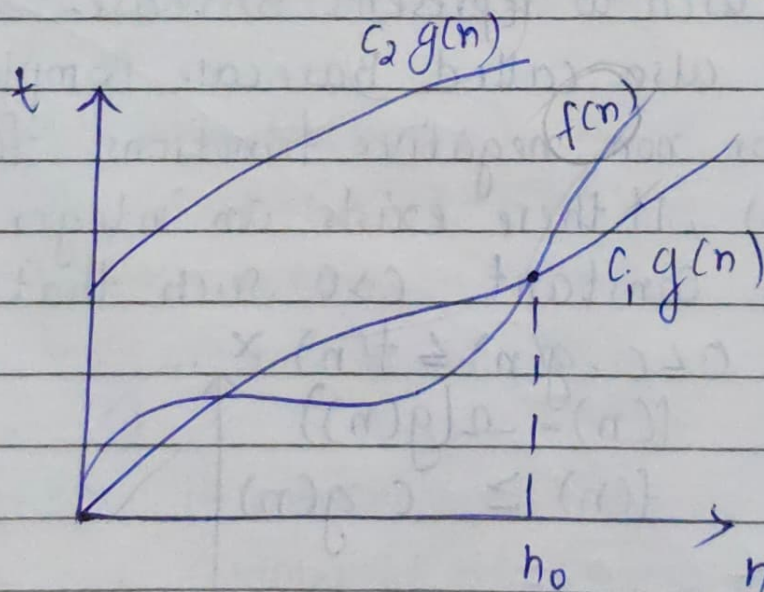
3.

Big Theta (Θ) Notation

- It is used to represent an average case.
- The lower and upper bound of the fn is provided by a notation.
- for non negative fn $f(n)$ & $g(n)$, if there exists an integer n_0 and two constants c_1 and c_2 , such that for all integer $n \geq n_0$

$$c_1 g(n) \leq f(n) \leq c_2 g(n)$$

lower bound
upper bound



4. Little oh Notation (o)

- Tightless upper bound notations.
- For non negative $f(n)$ and $g(n)$
If there exists an integer n_0 and a positive constant C , Such that for all integer $n \geq n_0$

$$0 \leq f(n) < C \cdot g(n)$$
$$f(n) = o(g(n))$$

$$\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = 0$$

5. Little Omega Notation (ω)

- Tightless lower bound Notations.
- For non negative function $f(n)$ & $g(n)$.
If there exists an integer n_0 and +ve constant C , Such that for all Integer.

$$0 \leq f(n) < C \cdot g(n)$$
$$f(n) = \omega(g(n))$$

$$\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = \infty$$

properties of Asymptotic Notation

1. Symmetric property

- $f(n) = O(g(n))$ is symmetric if and only if $g(n) = O(f(n))$
- $f(n) = \Theta(g(n))$ is symmetric iff $g(n) = \Theta(f(n))$
- $f(n) = \omega(g(n))$ is symmetric iff $g(n) = \omega(f(n))$.

2. Transpose property

- Applicable in worst case & Best case.
- Not in average case

$$f(n) = O(g(n)) \text{ iff } g(n) = \omega(f(n))$$

$\uparrow$
 worst
 case

$\uparrow$
 Best
 case

- also little Oh & little Omega.

$$f(n) = O(g(n)) \text{ iff } g(n) = \omega(f(n))$$

$\uparrow$
 upper bound

$\uparrow$
 lower bound

3. Transitivity property

- $f(n) = O(g(n))$ and $g(n) = O(h(n))$
then $f(n) = O(h(n))$.
- $f(n) = \Theta(g(n))$ and $g(n) = \Theta(h(n))$
then $f(n) = \Theta(h(n))$
- $f(n) = \Omega(g(n))$ and $g(n) = \Omega(h(n))$
then $f(n) = \Omega(h(n))$.

continue this with ω and θ .

GROWTH FUNCTION

It is the estimate no. of steps an algorithm process of

uses as its input growth.

- process of increasing no. of steps when input increases.

priority of order

less priority

linear time

logarithmic time.

$$O(1) < O(\log n) < O(n) < O(n \cdot \log n) \\ < O(n^2) < O(n^3) < O(2^n) < O(n!) \\ < O(n^n)$$

high priority

Q. Find Big O notation for the function

$$f(n) = 3n + 2$$

ans:-

$$0 \leq f(n) \leq c \cdot g(n) \quad \forall n \geq n_0$$

$$g(n) = n$$

$$c = 3 + 1 \text{ (upper bound)}$$

$$= 4$$

$$3n + 2 \leq 4n$$

$$\Rightarrow n = 1$$

$$3 + 2 \leq 4$$

$$5 \leq 4$$

$$\text{false} \Rightarrow 3n + 2 \leq 4 \cdot n \quad \forall n \geq n_0$$

$$\Rightarrow n = 2$$

$$3 \times 2 + 2 \leq 4 \times 2$$

$$8 \leq 8 \text{ True}$$

$$\Rightarrow n = 0$$

$$3 \times 0 + 2 \leq 4 \times 0$$

$$2 \leq 0 \text{ False}$$

$$\Rightarrow n = 3$$

$$3 \times 3 + 2 \leq 4 \times 3$$

$$11 \leq 12 \text{ True.}$$

$$n_0 = 2$$

$$f(n) = O(g(n))$$

$$f(n) = O(n)$$

$$3n + 2 \leq 4n \quad \forall n \geq 2$$

Q. Find the upper bound for the fn
 $f(n) = 5n^3 + n^2 + 6n + 2$

ans:- $0 \leq f(n) \leq c \cdot g(n) \forall n \geq n_0$

$$g(n) = n^3$$

$$c = 5 + 1 = 6 //$$

$$\Rightarrow 5n^3 + n^2 + 6n + 2 \leq 6n^3$$

$$1. n=0 \Rightarrow 2 \leq 0$$

$$2. n=1 \Rightarrow 5 + 1 + 6 + 2 \leq 6n^3$$

$$14 \leq 6$$

$$3. n=2 \Rightarrow 5 \times 2^3 + 2^2 + 6 \times 2 + 2 \leq 6 \times 2^3$$

$$40 + 4 + 12 + 2 \leq 48$$

$$n=3 \Rightarrow 5 \times 3^3 + 3^2 + 6 \times 3 + 3 \leq 6 \times 3^3$$

$$155 \leq 162$$

$$n=4 \Rightarrow 5 \times 4^3 + 4^2 + 6 \times 4 + 4 \leq 6 \times 4^3$$

$$f(n) = O(g(n))$$

$$\Rightarrow f(n) = O(n^3)$$

$$\Rightarrow \underline{5n^3 + n^2 + 6n + 2} \leq 6n^3 \forall n \geq 4$$

Q. Find lower bound for $f(n) = 27n^2 + 16n + 25$

ans:-

$$0 \leq c \cdot g(n) \leq f(n)$$

$$f(n) = \underline{2} (g(n))$$

$$g(n) = n^2$$

$$c = 27 \text{ (lower bound)}$$

$$\Rightarrow 27n^2 \leq 27n^2 + 16n + 25$$

$$n=0 \Rightarrow 0 \leq 25 \text{ (True)}$$

$$f(n) = O(g(n))$$

$$f(n) = O(n^2)$$

$$27n^2 \leq 27n^2 + 16n + 25 \quad \forall n \geq 0$$

Q. Find higher bound for fn $f(n) = 3n + 2$
ans:-

$$\underbrace{c_1 g(n)}_n \leq f(n) \leq \underbrace{c_2 g(n)}_0 \quad \forall n \geq n_0$$

$$g(n) = n$$

$$c_1 = 3 \text{ (lower)}$$

$$c_2 = 3 + 1 = 4$$

$$3n \leq 3n + 2 \leq 4n$$

$$n=1 \Rightarrow 3 \leq 3 + 2 \leq 4$$

$$3 \leq 5 \leq 4 \text{ False}$$

$$n=2 \Rightarrow 3 \times 2 \leq 3 \times 2 + 2 \leq 4 \times 2$$

$$6 \leq 8 \leq 8 \text{ (True)}$$

$$f(n) = O(g(n))$$

$$f(n) = O(n)$$

$$\Rightarrow 3n \leq 3n + 2 \leq 4n \quad \forall n \geq 2$$

Q. prove that $n! = O(n^n)$

ans:-

$$\text{If } \lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = 0 \quad f(n) = O(g(n))$$

$$\lim_{n \rightarrow \infty} \frac{n!}{n^n} = 0$$

$$\lim_{n \rightarrow \infty} \frac{n \left[1 \times \left(1 - \frac{1}{n}\right) \times \left(1 - \frac{2}{n}\right) \dots \times \frac{1}{n} \right]}{n^n}$$

$$= \left[1 \times \left(1 - \frac{1}{n}\right) \times \left(1 - \frac{2}{n}\right) \times \dots \times \frac{1}{n} \right]$$

Q. prove that $n^2 + n = O(n^3)$

ans:-

$$n^2 + n = f(n) \leq c \cdot g(n)$$

$$f(n) = n^2 + n$$

$$g(n) = n^3$$

$$f(n) \leq c \cdot g(n)$$

$$n^2 + n \leq c \cdot n^3$$

$$n=1 \Rightarrow c=1 \Rightarrow 2 \leq 1 \quad \text{False}$$

$$n=1, c=2 \quad 2 \leq 2 \quad \text{True}$$

$$n=2 \quad c=1 \quad 6 \leq 16 \quad \text{True}$$

$$n^2 + n = O(n^3), \quad \forall n \geq 1$$

$$c=2$$

Q. If $2^{2n} = O(2^n)$

ans:-

$$\begin{aligned} 2^{2n} &= 2^n \cdot 2^n \\ 2^n \cdot 2^n &= 2^{n+n} \\ &= 2^{2n} \end{aligned}$$

$$2^n \cdot 2^n = f(n) \leq c \cdot g(n)$$

$$f(n) = 2^n \cdot 2^n$$

$$g(n) = 2^n$$

$$2^n \cdot 2^n \leq c \cdot 2^n$$

$$2^n \leq c$$

In this fn, it depends on the value of n . So, we cannot define the value of c .

RECURRENCE (Recursive Fn)

1. Iteration - ④
2. Substitution - ②
3. Master's Method - ①
4. Recursive tree - ③

void test(int n) $\rightarrow T(n)$

{

if ($n > 0$) $\rightarrow 1$

{


```

print n → 1
test (n-1) → T(n-1)
}
}

```

$$T(n) = T(n-1) + 2 \rightarrow T(n-1) + 1$$

$$T(n) = \begin{cases} 1 & n=0 \\ T(n-1) + 1 & n>0 \end{cases}$$

$$T(n) = T(n-1) + 1 \text{ --- ①}$$

$$\begin{aligned} T(n-1) &= T(n-1-1) + 1 \\ &= T(n-2) + 1 \text{ --- ②} \end{aligned}$$

② in ①

$$\begin{aligned} T(n) &= [T(n-2) + 1] + 1 \\ &= T(n-2) + 2 \\ &= T(n-3) + 3 \end{aligned}$$

Continue for k times.

$$T(n) = T(n-k) + k$$

Assume, $n-k=0$, then $n=k$ Substitution Method

$$\begin{aligned} T(n) &= T(n-n) + n \\ &= T(0) + n \\ &= 1 + n \end{aligned}$$

$$\therefore \underline{\underline{T(n) = O(n)}}$$

2. Void test('n+h) $\rightarrow T(n)$

```

{
  if (n > 0)  $\rightarrow 1$ 
  {
    for (i=0; i < n; i++)  $\rightarrow n+1$ 
      print i  $\rightarrow n$ 
      test (n-1)  $\rightarrow T(n-1)$ 
  }
}

```

$$\begin{aligned}
 T(n) &= 1 + n + 1 + T(n-1) + n \\
 &= 2n + T(n-1) + 2 \\
 &= T(n-1) + n
 \end{aligned}$$

$$T(n) = \begin{cases} 1 & n=0 \\ T(n-1) + n & n>0 \end{cases} \Rightarrow \text{Recurrence eqn}$$

$$T(n) = T(n-1) + n \quad \text{--- (1)}$$

$$\begin{aligned}
 T(n-1) &= T(n-1-1) + n-1 \\
 &= T(n-2) + n-1 \quad \text{--- (2)}
 \end{aligned}$$

$$T(n-2) = T(n-3) + n-2$$

② 'n' ①

$$\begin{aligned}
 T(n) &= T(n-2) + (n-1) + n \\
 &= T(n-3) + (n-2) + (n-1) + n
 \end{aligned}$$

k times

$$= T(n-n) + [n-(n+1)] [n-(n+2)] + \dots + (n-1) + n$$

$n-k=0$

$$h=k$$

$$\Rightarrow \underbrace{T(0)}_1 + \underbrace{1+2+3+\dots+(n-1)+n}_{\frac{n(n+1)}{2}}$$

$$\Rightarrow 1 + \frac{n^2+n}{2}$$

$$\Rightarrow \underline{O(n^2)}$$

Q. $T(n) = T(n-1) - n^4$

Q. $T(n) = 2T(n-1) + 1$ (Hint $n-k=1$)

Q. $T(n) = n + 2T(n/2) - (1)$

$$T(n/2) = n/2 + 2T(n/4) - (2)$$

$$T(n/4) = n/4 + 2T(n/8) - (3)$$

$$T(n) = n + 2(n/2 + 2T(n/4)) - (4)$$

$$(2) \text{ in } (1) \Rightarrow$$

$$= n + n + 4T(n/4)$$

$$= 2n + 4T(n/4) - (4)$$

$$= 2n + 4(n/4 + 2T(n/8)) \text{ (3) in (4)}$$

$$= 2n + n + 8T(n/8)$$

$$= 3n + 8T(n/8)$$

:

$$T(n) = kn + 2^k T(n/2^k) - (5)$$

$$\frac{n}{2^k} = 1$$

$$n = 2^k$$

$$\log n = \log 2^k$$

$$= k \log 2$$

$$\log n = k \quad \text{--- (6)}$$

$$(6) \text{ in (5)} \Rightarrow T(n) = \log n \cdot n + 2^{\log n} T(1)$$

$$= n \log n + 2^{\log n}$$

$$= n \log n + n \log^2 2$$

$$= n \log n + n$$

$$a^{\log_b c} = b^{\log_a c}$$

$$\underline{T(n)} = O(n \log n)$$

Q.

$$T(n) = 4T(n/2) + n^2 \quad \text{--- (1)}$$

$$T(n/2) = 4T(n/4) + \frac{n^2}{4} \quad \text{--- (2)}$$

$$T(n/4) = 4T(n/8) + \frac{n^2}{16} \quad \text{--- (3)}$$

$$T(n) = 4T(n/2) + n^2$$

$$\text{② in ①} \Rightarrow$$

$$= 4 \times (4T(n/4) + \frac{n^2}{4}) + n^2$$

$$= 16T(n/4) + n^2 + n^2$$

$$= 16T(n/4) + 2n^2 \quad \text{--- (4)}$$

$$= 16(4T(n/8) + \frac{n^2}{16}) + 2n^2$$

$$(3) \text{ in (4)}$$

$$= 64T(n/8) + n^2 + 2n^2$$

$$= 64T(n/8) + 3n^2$$

$$= 4^k T(n/2^k) + k n^2 \quad \text{--- (5)}$$

$$4k \cdot \frac{n}{2^k} = 1$$

$$n = 2^k$$

$$\log n = k \quad (6)$$

(6) in (5)

$$T(n) = 4 \log^k T(1) + \log n n^2$$

$$= n \log^4 + n^2 \log n$$

$$= n^2 + n^2 \log n$$

$$T(n) = O(n^2 \log n)$$

H.W

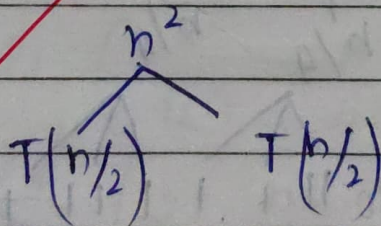
Q.

$$T(n) = \begin{cases} 2^2 T(n/2) + 2n + 3 & n > 1 \\ 2 & n = 1 \end{cases}$$

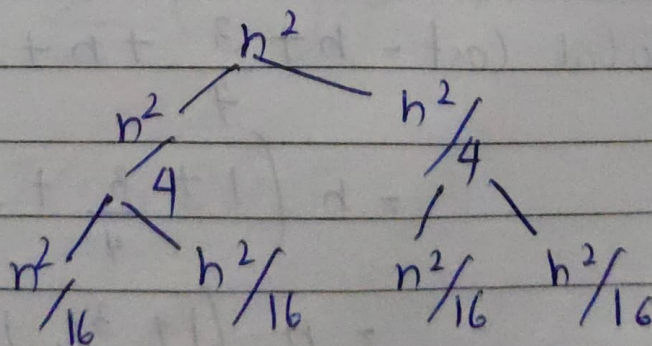
Q.

$$T(n) = 2T(n/2) + n^2$$

$$T(n/2) = 2T(n/4) + n^2/4$$



$$T(n/4) = 2T(n/8) + n^2/16$$



$$T(n) = \frac{n^2}{2} + \frac{n^2}{2^2} + \frac{n^2}{2^3} + \dots + \frac{n^2}{2^k}$$

$$= n^2 \left[1 + \frac{1}{2} + \frac{1}{2^2} + \dots + \frac{1}{2^k} \right]$$

$$= n^2 \left[\frac{1 \times \left(\left(\frac{1}{2} \right)^{k+1} - 1 \right)}{\frac{1}{2} - 1} \right]$$

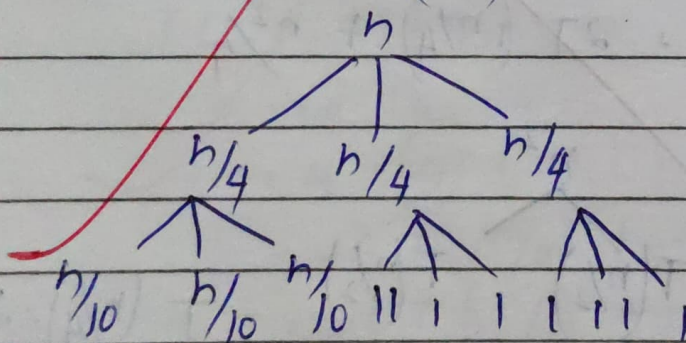
$$\frac{n}{2^k} = 1$$

$$n = 2^k$$

$$\log n = k$$

$$T(n) = \underline{\underline{4 \log n}}$$

Q. $T(n) = 3T(n/4) + n$



$$\text{Total Cost} = n + \frac{3n}{4} + n + \frac{9n}{16} + \dots$$

$$= n \left(1 + \frac{3}{4} + \frac{9}{16} + \dots \right)$$

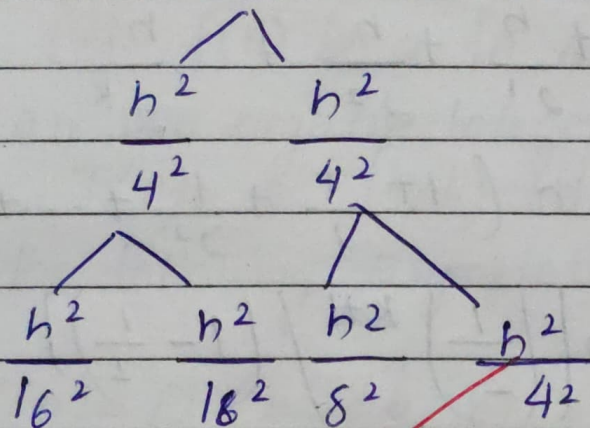
$$= n \left(1 + \frac{3}{4} + \frac{3^2}{4^2} + \dots \right)$$

$$= h \left(\frac{1}{1-3/4} \right)$$

$$= \underline{\underline{4h = O(n)}}$$

$$T\left(\frac{n}{2}\right) = T\left(\frac{n/2}{4}\right) + \frac{n/2}{2} + \frac{n/2}{2}$$

Q. $T(n) = T\left(\frac{n}{4}\right) + T\left(\frac{n}{2}\right) + n^2$



$$2i \left(\frac{n}{4i} \right) \text{ and } 2i \left(\frac{n}{2i} \right)$$

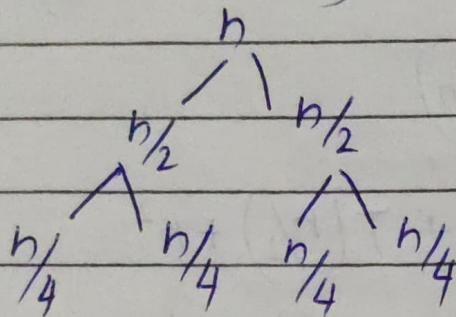
$$= 2i \left(\frac{n}{16i} \right) + 2i \left(\frac{n^2}{4i} \right)$$

$$= 2i \left(\frac{n^2}{16i} + \frac{n^2}{4i} \right)$$

$$T(n) = \sum_{i=0}^{\log_4 n} \left(2i \frac{n^2}{16i} + 2i \frac{n^2}{4i} \right)$$

$$\underline{\underline{T(n) = O(n^2)}}$$

Q. $T(n) = 2T(n/2) + n$



$$\frac{n}{2^0}$$

$$\frac{n}{2^1}$$

$$\frac{n}{2^2}$$

$$T(n) = n + \frac{n}{2^1} + \frac{n}{2^2} + \dots + \frac{n}{2^k}$$

$$= n \left(1 + \frac{1}{2} + \frac{1}{2^2} + \dots + \frac{1}{2^k} \right)$$

$$= n \left[\left(\frac{1}{2} \right)^{k+1} / \left(1 - \frac{1}{2} \right) \right]$$

$$= n \sum_{i=0}^k \frac{1}{2^i} = n \times 1$$

$$T(n) = n$$

$$= \underline{\underline{O(n)}}$$

MASTER'S THEOREM

Master's Method provides a cookbook method for solving the recurrence of the form

$$T(n) = a \cdot T(n/b) + f(n)$$

Theorem :-

Let $a \geq 1$ and $b > 1$ be constants.

Let $f(n)$ be a fn and $T(n)$ be defined as non-negative integers by recurrence.

$T(n) = a \cdot T(n/b) + f(n)$ then $T(n)$ has the following asymptotic bounds

Q1. $f(n) = O(n^{\log_b a - \epsilon})$; $g(n) > f(n)$
 $T(n) = \Theta(n^{\log_b a})$

2. $f(n) = \Theta(n^{\log_b a})$; $g(n) = f(n)$
 $T(n) = (\Theta(\log^k(n)) n^{\log_b a})$

3. $f(n) = \Omega(n^{\log_b a + \epsilon})$; $g(n) < f(n)$
 $= \Theta(f(n))$

Q. Solve the recurrence $T(n) = 9 \cdot T(n/3) + n$

$a = 9$ and $b = 3$

$f(n) = n$

$g(n) = n^{\log_b a}$

$$= n \log_3^9$$

$$= n^2$$

By using master's theorem, we use 1st eqn;

$$\underline{\underline{T(n) = \Theta(n^2)}}$$

Here $f(n) = n$
 $g(n) = n^2$

$n^2 > n$, so
we use 1st.

2. $T(n) = T(2n/3) + 1$

Here $a = 1$

$$b = 2n/3 = n/3/2$$

$$\text{So, } b = 3/2$$

$$f(n) = 1$$

$$g(n) = n \log^a b$$

$$= n^0 \log_{3/2}^1 = 0$$

$$= \Theta(1)$$

$$g(n) = f(n) \text{ so,}$$

By using master's theorem, we use 11th eqn

$$\underline{\underline{T(n) = \Theta(n)}}$$

3. $T(n) = 3T(n/4) + n \log n$

$$a = 3$$

$$b = 4$$

$$f(n) = n \log n$$

$$g(n) = n \log^a b$$

$$= n \log^3 4$$

$$=$$

$$T(n) = \underline{\underline{O(n \log n)}}$$

Q. $T(n) = 0.5 T(n/2) + n$

$$a = 0.5$$

$$b = 2$$

$$f(n) = n$$

$$g(n) = n \log^a_b$$

$$= n \log^{0.5}_2$$

For applying master's theorem a should be greater than 1. So this is Invalid.

Q. $T(n) = 2^n T(n/2) + n^n$

$a = 2^n$ is not constant, so we can't apply master's theorem.

Q. $T(n) = 5 T(n/2) - n \log n$

Here $f(n)$ is a negative $f(n)$, so we can't apply master's theorem.

Extension for Master's Theorem

$$T(n) = a T(n/b) + f(n)$$

$$f(n) = \Theta(n \log_b^a \log^k(n))$$

1. If $k > -1$ then $\Theta(n \log_b^a \log^{(k+1)}(n))$

2. If $k = -1$ then $\Rightarrow \Theta(n \log_b^a \log(\log(n)))$

3. $k < -1 \Rightarrow \Theta(n \log_b^a)$

H.W^Q ans:-
$$T(n) = \begin{cases} 2 & n=1 \\ 2T(n/2) + 2n + 3 & n > 1 \end{cases}$$

ans:-

$$T(n) = 2T(n/2) + 2n + 3$$

$$T(n/2) = 2T(n/2 \times 2) + 2 \times (n/2) + 3$$

$$T(n/2) = 2T(n/4) + 2(n/2) + 3$$

$$= 2T(n/4) + 2n/2 + 3$$

$$= 2T(n/4) + n + 3$$

$$T(n) = 4 [2T(n/4) + n + 3] + 2n + 3$$

$$= 8T(n/4) + 4n + 12 + 2n + 3$$

$$= 8T(n/4) + 6n + 15$$

$$T(n/4) = 2T(n/2 \times 2) + 2(n/4) + 3$$

$$= 2T(n/8) + 2n/4 + 3$$

$$= 2T(n/8) + n/2 + 3$$

$$\begin{aligned}
 T(n) &= 4 \cdot 2T(n/8) + n/2 + 3 \\
 &= 8T(n/8) + 4n/2 + 3 \\
 &= 8T(n/8) + 2n + 3 \\
 &= 2^k T(n/2^k) + 2n + 3
 \end{aligned}$$

$$\begin{aligned}
 \frac{n}{2^k} &= 1 \\
 n &= 2^k
 \end{aligned}$$

$$\log n = 2^k$$

$$\log n = k$$

$$\begin{aligned}
 T(n) &= 2^k T(n/2^k) + 2 \log n + 3 \\
 &= 2^{\log n} T(n/2^{\log n}) + 2 \log n + 3 \\
 &= n^{\log 2} T(n/n \log 2) + 2 \log n + 3 \\
 &= n^1 T(n/n) + 2 \log n + 3 \\
 &= n + 2 \log n + 3 \\
 T(n) &= \underline{O(\log n)}
 \end{aligned}$$

8/11/2025